```
var_dump(octdec('0o3301'));
var_dump(hexdec('6C1'));
var_dump(hexdec('0x6C1'));
```

In all cases the output will be `int(1729)`. Using the PHP literal notation prefixes is fine, but optional. Be warned that these three functions will accept invalid characters, which will be ignored (and generate a deprecation notice in PHP 7.4 or later). Handling invalid characters in this way will be removed from a future version of PHP, so consider yourselves suitably warned:

```
var_dump(hexdec('Z6c1')); // deprecated in PHP >= 7.4
```

```
Output:
Deprecated: Invalid characters passed for attempted conversion, these have
been ignored
int(1729)
```

As previously mentioned, arbitrary number bases up to Base 36 can also be handled. The built-in function `base_convert()` accepts a string representation of a number in any base and will return the representation in a chosen target base. For example, `base_convert('z', 36, 10);` would return `string(2) "35"`:

```
var_dump(base_convert('11011000001', 2, 10));
var_dump(base_convert('0o3301', 8, 10));
var_dump(base_convert('6c1', 16, 10));
```

```
Output:
string(4) "1729"
```

Again, any invalid characters present in the source string are ignored with a deprecation notice.

Formatting

The representation of numbers is limited internally in PHP to integers and IEEE double-precision floating point (fully explained in the "Limits and Precision" section). However, there are many conventions when it comes to displaying numbers for human consumption, usually to make them easier to read or to meet certain conventions depending on locale or the problem domain (e.g., finance).